

# SuccessionPublicFormController Class

Public form controller for guest user access to succession pathway form. Uses URL parameters (caseId, accountId, etc.) to pre-fill form data. No token-based authentication - relies on URL parameter obscurity.

**Author** Estate Administration Team

**Date** 2025

## Class Diagram

```
graph TD
    SuccessionPublicFormController["SuccessionPublicFormController"]:::mainApexClass
    click SuccessionPublicFormController "/objects/SuccessionPublicFormController/"

    SuccessionPublicFormController_Test["SuccessionPublicFormController_Test"]:::apexTestClass
    click SuccessionPublicFormController_Test
    "/apex/SuccessionPublicFormController_Test/"

    SuccessionPublicFormController_Test --> SuccessionPublicFormController

    classDef apexClass fill:#FFF4C2,stroke:#CCAA00,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
    classDef apexTestClass fill:#F5F5F5,stroke:#999999,stroke-width:3px,rx:12px,ry:12px,shadow:drop,color:#333;
    classDef mainApexClass fill:#FFB3B3,stroke:#A94442,stroke-width:4px,rx:14px,ry:14px,shadow:drop,color:#333,font-weight:bold;

    linkStyle 0 stroke:#FF8C00,stroke-width:2px;
```

## Apex Code

```
/**
 * @description Public form controller for guest user access to succession pathway
form.
 *
 * Uses URL parameters (caseId, accountId, etc.) to pre-fill form data.
 * No token-based authentication - relies on URL parameter obscurity.
 * @author Estate Administration Team
 * @date 2025
 */
public with sharing class SuccessionPublicFormController {
    /**
     * @description Fetches case and related data for public form pre-filling
     * @param caseId Salesforce Case ID from URL parameter
     * @return FormData wrapper with case, account, financial account, and successor
info
     */
```

```

@AuraEnabled(cacheable=true)
public static FormData getFormData(String caseId) {
    try {
        if (String.isBlank(caseId)) {
            throw new AuraHandledException('Case ID is required');
        }

        // Query case with all related data including Person Account fields
        List<Case> cases = [
            SELECT
                Id,
                CaseNumber,
                Subject,
                Status,
                Type,
                AccountId,
                Account.Name,
                Account.PersonEmail,
                Account.Phone,
                Account.FirstName,
                Account.LastName,
                Account.IsPersonAccount,
                ContactId,
                Contact.Name,
                Contact.Email,
                Contact.Phone,
                Successor__c,
                FinServ__FinancialAccount__c,
                FinServ__FinancialAccount__r.Name,
                FinServ__FinancialAccount__r.FinServ__Balance__c,
                FinServ__FinancialAccount__r.FinServ__FinancialAccountNumber__c,
                RecordType.DeveloperName
            FROM Case
            WHERE
                Id = :caseId
                AND RecordType.DeveloperName = 'EstateAdministration'
                AND Type = 'Named Successor Enactment'
            WITH USER_MODE
            LIMIT 1
        ];

        if (cases.isEmpty()) {
            throw new AuraHandledException('Case not found or invalid case type');
        }

        Case c = cases[0];

        // CRITICAL FIX: Use Successor__c field (Contact lookup) for successor
        information
        // This field points to the actual successor Contact record
        Id successorLookupId = c.Successor__c;
    }
}

```

```

// Query successor Contact information directly
String successorName = null;
String successorEmail = null;
String successorPhone = null;

if (successorLookupId != null) {
    List<Contact> successorContacts = [
        SELECT Name, Email, Phone
        FROM Contact
        WHERE Id = :successorLookupId
        WITH USER_MODE
        LIMIT 1
    ];

    if (!successorContacts.isEmpty()) {
        successorName = successorContacts[0].Name;
        successorEmail = successorContacts[0].Email;
        successorPhone = successorContacts[0].Phone;
    }
}

// Query successor financial account role (only if successorLookupId exists)
List<FinServ__FinancialAccountRole__c> successorRoles = new
List<FinServ__FinancialAccountRole__c>();

if (successorLookupId != null) {
    successorRoles = [
        SELECT
            Id,
            FinServ__Role__c,
            SuccessorAllocation__c,
            FinServ__RelatedContact__c,
            FinServ__RelatedContact__r.Name,
            FinServ__RelatedContact__r.Email,
            FinServ__RelatedContact__r.Phone
        FROM FinServ__FinancialAccountRole__c
        WHERE
            FinServ__FinancialAccount__c = :c.FinServ__FinancialAccount__c
            AND FinServ__Role__c LIKE '%Successor%'
            AND FinServ__RelatedContact__c = :successorLookupId
        WITH USER_MODE
        LIMIT 1
    ];
}

// Build response wrapper
FormData data = new FormData();
data.caseId = c.Id;
data.caseNumber = c.CaseNumber;
data.accountName = c.Account != null ? c.Account.Name : null;
data.successorName = successorName;
data.successorEmail = successorEmail;

```

```

data.successorPhone = successorPhone;

if (c.FinServ__FinancialAccount__c != null) {
    data.financialAccountName = c.FinServ__FinancialAccount__r.Name;
    data.financialAccountNumber =
c.FinServ__FinancialAccount__r.FinServ__FinancialAccountNumber__c;
    data.accountBalance = c.FinServ__FinancialAccount__r.FinServ__Balance__c;
}

if (!successorRoles.isEmpty()) {
    data.allocationPercentage = successorRoles[0].SuccessorAllocation__c;
}

return data;
} catch (Exception e) {
    System.debug('Error in getFormData: ' + e.getMessage());
    throw new AuraHandledException(
        'Error loading form data: ' + e.getMessage()
    );
}
}

/**
 * @description Saves pathway selection and form data submitted by successor
 * @param caseId Salesforce Case ID
 * @param pathwaySelection Selected pathway (Final Grant, New DAF, Disclaim)
 * @param formData JSON string with additional form fields
 * @return Success message
 */
@AuraEnabled
public static String savePathwaySelection(
    String caseId,
    String pathwaySelection,
    String formData
) {
    try {
        if (String.isBlank(caseId)) {
            throw new AuraHandledException('Case ID is required');
        }

        if (String.isBlank(pathwaySelection)) {
            throw new AuraHandledException('Pathway selection is required');
        }

        // Query case to verify it exists and is valid
        List<Case> cases = [
            SELECT Id, Status, Pathway_Confirmed__c
            FROM Case
            WHERE
                Id = :caseId
                AND RecordType.DeveloperName = 'EstateAdministration'
                AND Type = 'Named Successor Enactment'

```

```

        WITH USER_MODE
        LIMIT 1
    ];

    if (cases.isEmpty()) {
        throw new AuraHandledException('Case not found or invalid case type');
    }

    Case c = cases[0];

    // Update case with pathway selection
    // Pathway_Confirmed__c is a picklist that stores the pathway choice
    // Valid values: "Not Selected", "Final Grant", "New DAF Account", "Disclaim
Assets"
    c.Pathway_Confirmed__c = pathwaySelection;
    c.Form_Completed_Date__c = Date.today();
    c.Status = 'Pathway Selection Received';

    // Parse additional form data if provided
    if (String.isNotBlank(formData)) {
        Map<String, Object> formFields = (Map<String, Object>)
JSON.deserializeUntyped(
        formData
    );

    // Map form fields to Case fields
    if (formFields.containsKey('additionalNotes')) {
        c.Description = (String) formFields.get('additionalNotes');
    }

    // Add more field mappings as needed based on pathway-specific requirements
    }

    Database.update(c, false, AccessLevel.USER_MODE);

    return 'Pathway selection saved successfully';
} catch (Exception e) {
    System.debug('Error in savePathwaySelection: ' + e.getMessage());
    throw new AuraHandledException(
        'Error saving pathway selection: ' + e.getMessage()
    );
}
}

/**
 * @description Wrapper class for form data response
 */
public class FormData {
    @AuraEnabled
    public String caseId;
    @AuraEnabled
    public String caseNumber;
}

```

```
@AuraEnabled
public String accountName;
@AuraEnabled
public String successorName;
@AuraEnabled
public String successorEmail;
@AuraEnabled
public String successorPhone;
@AuraEnabled
public String financialAccountName;
@AuraEnabled
public String financialAccountNumber;
@AuraEnabled
public Decimal accountBalance;
@AuraEnabled
public Decimal allocationPercentage;
}
}
```

## Methods

### getFormData(caseId)

AURAENABLED

Fetches case and related data for public form pre-filling

#### Signature

```
public static FormData getFormData(String caseId)
```

#### Parameters

| Name   | Type   | Description                           |
|--------|--------|---------------------------------------|
| caseId | String | Salesforce Case ID from URL parameter |

#### Return Type

##### FormData

FormData wrapper with case, account, financial account, and successor info

---

### savePathwaySelection(caseId, pathwaySelection, formData)

AURAENABLED

Saves pathway selection and form data submitted by successor

#### Signature

```
public static String savePathwaySelection(String caseId, String pathwaySelection,
String formData)
```

**Parameters**

| Name             | Type   | Description                                       |
|------------------|--------|---------------------------------------------------|
| caseId           | String | Salesforce Case ID                                |
| pathwaySelection | String | Selected pathway (Final Grant, New DAF, Disclaim) |
| formData         | String | JSON string with additional form fields           |

**Return Type**

**String**

Success message

**Classes**

**FormData Class**

Wrapper class for form data response

**Fields**

**caseId**

AURAENABLED

**SIGNATURE**

```
public caseId
```

**TYPE**

String

**caseNumber**

AURAENABLED

**SIGNATURE**

```
public caseNumber
```

**TYPE**

String

**accountName**

AURAENABLED

**SIGNATURE**

```
public accountName
```

**TYPE**

String

---

**successorName**

AURAENABLED

**SIGNATURE**

```
public successorName
```

**TYPE**

String

---

**successorEmail**

AURAENABLED

**SIGNATURE**

```
public successorEmail
```

**TYPE**

String

---

**successorPhone**

AURAENABLED

**SIGNATURE**

```
public successorPhone
```

**TYPE**

String

---

**financialAccountName**

AURAENABLED

**SIGNATURE**

```
public financialAccountName
```

**TYPE**

String

---

**financialAccountNumber**

AURAENABLED

**SIGNATURE**

```
public financialAccountNumber
```

**TYPE**

String

---

**accountBalance**

AURAENABLED

**SIGNATURE**

```
public accountBalance
```

**TYPE**

Decimal

---

**allocationPercentage**

AURAENABLED

**SIGNATURE**

```
public allocationPercentage
```

**TYPE**

Decimal